

GIT E VERSIONING

Appunti del mattino — 06 marzo 2026

1. Il Versionamento

Il versionamento (o version control) consente di salvare "istantanee" del progetto nel tempo, permettendo di:

- Tornare a versioni precedenti del codice
- Lavorare in parallelo su funzionalità diverse
- Collaborare in team senza sovrasciversi

2. Git

Git è un sistema di controllo versione distribuito (DVCS) creato da Linus Torvalds nel 2005.

2.1 Concetti fondamentali

Untracked (U)

La U sta per Untracked: Git ha rilevato che il file esiste nella cartella di lavoro, ma non lo sta ancora monitorando. Per Git quel file è invisibile finché non si esegue `git add` per portarlo nella staging area.

Staging Area

La staging area è una zona intermedia tra i file di lavoro e la storia definitiva del progetto. Quando si modifica un file, Git non lo salva automaticamente: bisogna prima "sceglierlo" con `git add`, che lo porta nella staging area. Solo dopo, con `git commit`, la modifica viene registrata ufficialmente nella storia del repository.

💡 `git add` corrisponde al pulsante + nella sezione **SOURCE CONTROL** di VS Code.

Hash del commit

L'hash è un codice esadecimale generato automaticamente da Git che identifica il commit in modo univoco — nessun altro commit nel repository avrà mai lo stesso hash. Di solito Git ne mostra una versione abbreviata (7 caratteri), ma internamente è un codice molto più lungo. Serve per riferirsi a uno specifico commit, ad esempio per tornare indietro a quel punto della storia.

Modified (M)

La M gialla accanto a un file sta per Modified: significa che il file era già tracciato da Git (aveva già un commit precedente), ed è stato modificato ma non ancora committato. È diversa dalla U — lì il file non era ancora conosciuto da Git, qui invece Git lo conosce già e ha rilevato delle variazioni rispetto all'ultimo commit.

3. Comandi Git principali

Comando	Descrizione
<code>git init</code>	Inizializza un nuovo repository nella cartella corrente
<code>git add <file></code>	Aggiunge un file specifico alla staging area
<code>git add .</code>	Aggiunge tutti i file modificati/nuovi alla staging area (esclusi quelli in .gitignore)
<code>git commit</code>	Registra le modifiche della staging area nella storia del repository
<code>git diff</code>	Mostra le differenze tra versioni di un file (rosso = rimosso, verde = aggiunto)
<code>git restore <file></code>	Scarta le modifiche non committate, riportando il file all'ultimo commit
<code>git restore --source <hash> <file></code>	Ripristina un singolo file alla versione di uno specifico commit
<code>git restore --staged <file></code>	Rimuove un file dalla staging area senza cancellarne le modifiche
<code>git blame <file></code>	Mostra chi ha scritto ogni riga del file, quando e in quale commit
<code>git branch</code>	Elenca tutti i branch esistenti, con asterisco su quello attivo
<code>git branch <nome></code>	Crea un nuovo branch senza spostarsi
<code>git switch <nome></code>	Si sposta su un branch esistente
<code>git merge <branch></code>	Fonde il branch indicato nel branch corrente
<code>git pull origin master</code>	Scarica l'ultima versione con le modifiche apportate
<code>git clone <url></code>	Clona un repository remoto in locale

4. Best Practice per i Commit

La regola generale è: committa spesso, dopo ogni modifica logicamente completa — non dopo ogni singola riga, ma nemmeno una volta a fine giornata con tutto insieme.

Un buon momento per committare è quando hai finito una cosa precisa:

- Hai aggiunto una funzione
- Hai corretto un bug
- Hai scritto un testo o una sezione

In questo modo, se qualcosa si rompe, sai esattamente dove tornare indietro e capisci subito cosa ha causato il problema.

5. Branch e Sviluppo Parallelo

Un branch è un percorso di sviluppo parallelo. La storia dei commit è una linea retta — a un certo punto ci si può "ramificare" e lavorare su una copia isolata del progetto, senza toccare il codice principale.

Branch di sviluppo vs Master

Il branch di sviluppo è dove si lavora liberamente — si aggiungono file, si sperimentano funzionalità, si rischia di rompere qualcosa. Il master (o main) invece rappresenta la versione stabile e funzionante del progetto, quella che in un contesto professionale sarebbe già in produzione.

Flusso tipico con i branch

1. Si sviluppa sul branch secondario
2. Si testa il codice
3. Quando tutto è verificato, si fa il merge su master

In questo modo master non contiene mai codice rotto o incompleto.

Workflow operativo

`master → dev → si scrive codice, si fanno test → push su dev → switch su master → merge`

Comandi per il merge:

4. `git switch master` — ci si sposta sul branch master (quello che "riceve" la fusione)
5. `git status` — si controlla di avere tutto pulito prima del merge
6. `git merge dev-line` — si fonde il branch dev-line dentro master

Dopo il merge, master conterrà tutti i commit di entrambi i branch.

6. Git in VS Code

Initialize Repository in VS Code esegue `git init` in locale, creando la cartella nascosta `.git` che permette a Git di iniziare a tracciare il progetto.

Publish to GitHub invece fa il lavoro opposto: crea il repository remoto su GitHub e ci carica tutto, eseguendo sostanzialmente un `git remote add origin <url>` seguito da un `git push`.

7. GitHub e Lavoro in Team

GitHub è un repository condiviso. Il merge avviene su GitHub (in remoto). Origin è il repository a cui si è collegati (quello remoto).

Flusso di lavoro in team

7. Clone (1 volta): `git clone <link repo>`
8. Ogni mattina / alla fine di ogni lavoro: `git pull` — per lavorare sempre con il codice più recente
9. Sviluppo su branch DEV (nuova feature) → `commit` e `push` sul proprio branch
10. Non è detto che si faccia pull request ogni giorno — si può ancora stare sviluppando
11. Una volta terminato lo sviluppo in DEV si fa la pull request, e si fa il merge

Regole d'oro

- ⚠ *Fare sempre backup manuali → LO ZIPPONE!*
- ⚠ *Non pushare mai direttamente su MASTER (si può impedire da GitHub).*
- 💡 *Ogni mattina bisogna fare PULL perché potrebbe aver mergiato qualcos'altro nella notte.*

8. Note Progetto Angular

Obiettivo: avere le funzioni CRUD su tutte le entità, senza sicurezza per ora.

Entità principali:

- Inserire Post
- Definire Blog
- Bannare utenti e affini

Piano di sviluppo

12. API – CRUD: se abbiamo entità, abbiamo le API
13. Appena abbiamo le API si implementa la security
14. Test + DB di prova

Scadenza: per lunedì sera — tutto il backend funzionante con security e test.

- 💡 *Cercare un componente Angular esistente. Suddividere tutto in pezzi, trovando soluzioni furbe.*